SPRINTS • AI ENGINEERING PROTOTYPE

Multi-Agent AI Career & Assessment Orchestrator

Comprehensive Technical and Implementation Report (Release v1.1.0)

Status: Hardened Working Prototype	**Orchestration:** LangGraph 1.2+ (StateGraph)
Language: Python 3.10.8	**LLM:** Google Gemini via LangChain
Learner DB: SQLite (Safe Parameterized Ops)	**Sandbox:** Docker (sprints-python-sandbox:latest)
Checkpointer: SqliteSaver (Durable Local Storage)	**Automated CI:** 47 / 47 Pytest Suite Passing (100%)

Project status: Working prototype\
Primary language: Python 3.10.8\
Orchestration: LangGraph\
LLM provider: Google Gemini through LangChain\
Vector database: ChromaDB\
Learner database: SQLite\
Code execution: Docker sandbox\
Test framework: pytest

---

1. Project Overview

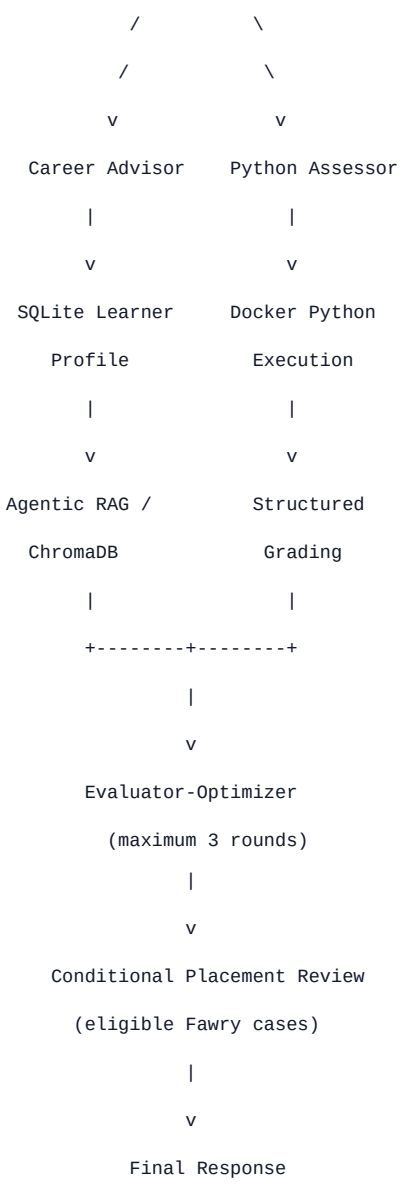
The Sprints Multi-Agent AI Career & Assessment Orchestrator is a Python prototype that routes learner requests to specialized agents. It currently supports two main workflows:

1. **Career Advisor:** retrieves a learner profile from SQLite, searches project knowledge in ChromaDB, evaluates retrieval quality, and generates structured career recommendations.
2. **Python Assessor:** executes submitted Python code in a restricted Docker container and evaluates the result against an assignment.

A central LangGraph workflow coordinates routing, agent execution, response evaluation/optimization, and a conditional human review step for selected placement recommendations.

The project is a prototype. It does not connect to a live jobs or vacancies database, does not contact employers, and does not submit job applications.

2. High-Level Architecture



The graph is implemented with **StateGraph(AgentState)**. A persistent **SqliteSaver** checkpointer is attached by default when the graph is compiled, saving thread checkpoints and writes to **data/orchestrator_checkpoints.sqlite3**. An **InMemorySaver** is also supported for ephemeral testing.

3. Technology Stack

The versions below are taken from the project's active virtual environment using **python -m pip freeze**. Python 3.10.8 is the interpreter version previously reported for this environment.

Component Version

--- ---

Python 3.10.8

LangChain 1.4.7

LangGraph 1.2.18

LangChain Core 1.7.3

LangChain Google GenAI 4.4.2

LangChain Chroma 1.1.0

LangChain HuggingFace 1.2.3

LangChain Text Splitters 1.1.3

ChromaDB 1.5.9

Pydantic 2.14.2

Sentence Transformers 6.2.0

Transformers 5.20.2

PyTorch 2.14.1

NumPy 2.2.6

Pandas 2.3.3

Google GenAI SDK 2.33.0

python-dotenv 1.2.4

python-pptx 1.0.2

pypdf 6.21.4

pytest 9.2.0

The complete environment snapshot is stored in the root **requirements.txt**. It contains all packages from the active environment, including transitive dependencies.

4. Repository Structure

The public repository contains source code, tests, configuration, and documentation. Local datasets, client-provided source materials, learner records, SQLite databases, and ChromaDB persistence files are intentionally excluded.

```
sprints-ai-orchestrator/
■■■ .github/
■   ■■■ workflows/
■       ■■■ tests.yml
■■■ sandbox/
■   ■■■ Dockerfile
■■■ src/
■   ■■■ agents/
■   ■   ■■■ advisor_agent.py
■   ■   ■■■ assessor_agent.py
■   ■   ■■■ evaluator_optimizer.py
■   ■   ■■■ orchestrator.py
■   ■■■ tools/
■   ■   ■■■ db_queries.py
■   ■   ■■■ docker_executor.py
■   ■   ■■■ rag_retriever.py
■   ■   ■■■ sqlite_checkpoint.py
■   ■   ■■■ test_runner.py
```

```
■   ■■■ state.py
■   ■■■ llm.py
■   ■■■ main.py
■■■ docs/
■   ■■■ DEMO_SCRIPT.md
■   ■■■ TECHNICAL_DOCUMENTATION.md
■■■ tests/
■   ■■■ test_agents/
■   ■   ■■■ test_agentic_rag.py
■   ■   ■■■ test_assessor_agent.py
■   ■   ■■■ test_evaluator_optimizer.py
■   ■   ■■■ test_orchestrator_evaluator.py
■   ■■■ test_tools/
■       ■■■ test_db_queries.py
■       ■■■ test_docker_executor.py
■       ■■■ test_sqlite_checkpoint.py
■       ■■■ test_test_runner.py
■■■ LICENSE
■■■ pytest.ini
■■■ requirements.txt
■■■ README.md
■■■ .gitignore
```

The following runtime inputs are expected to be provisioned locally and are not distributed in this public repository:

- Approved curriculum or project-context documents used for ingestion.
- Locally generated processed text and ChromaDB data.
- The SQLite learner database and any learner export files.
- A local **.env** file containing the Gemini API credential.

Do not add client recordings, client presentations, transcripts, learner exports, or identifiable learner records to the public repository. Confirm ownership, consent, and publication permission before sharing any source material.

5. Shared State and Structured Models

src/state.py defines the shared **AgentState** TypedDict used by

LangGraph. It includes fields for:

- Conversation messages and request routing.
- Learner ID and learner profile.
- Retrieved context, source metadata, and retrieval sufficiency.
- Submitted code, assignment, and expected output.
- Grading result and career advice.
- Evaluator feedback and iteration count.
- Human approval requirement, decision, and approval details.
- Errors and final response.

The state module also defines Pydantic models:

`GradingFeedback`

- **score**: integer from 0 to 100.
- **feedback_points**: one or more feedback items.
- **pass_status**: Boolean pass/fail result.

`CareerAdvice`

A structured career response containing the learner name, current track, skill assessment, recommended roles, skill gaps, learning roadmap, salary insight, and overall recommendation.

Using structured output helps keep agent responses aligned with the application's expected data shape.

6. Career Advisor Workflow

The Career Advisor implementation is in **src/agents/advisor_agent.py**.

6.1 Learner profile retrieval

The advisor first calls **get_learner_by_id()** to retrieve the learner record from SQLite. If the ID is missing or no matching record exists, the function returns an **Unable to verify**-style error rather than inventing a profile.

The database track field **bootcamp_track** is normalized into **current_track** for downstream use.

6.2 Agentic RAG

The advisor searches ChromaDB through **retrieve_context()** with a top-k value of 4. Each returned item retains its source and trust-level metadata.

The retrieval process uses three possible attempts: 1. Initial retrieval. 2. First retry after query reformulation. 3. Second retry after another reformulation, if needed.

After each retrieval, a structured retrieval evaluator assesses relevance and evidence sufficiency. If the evidence is insufficient, a query reformulator creates a more focused query based on the missing information. The trace records the query, sources, document count, sufficiency, relevance score, reasoning, and missing information for each attempt.

The maximum is one initial attempt plus two retries (**MAX_RETRIEVAL_RETRIES = 2**).

6.3 Career recommendation generation

When retrieval is sufficient, the advisor passes the verified learner profile and retrieved context to a Gemini model configured for

structured **CareerAdvice** output.

The prompt explicitly instructs the model to: - Use the SQLite learner profile as the source of learner facts. - Avoid inventing skills, qualifications, certifications, experience, or assessment results. - Identify skill gaps only when supported by evidence. - Avoid treating project requirements as official employer hiring requirements. - Avoid claiming that an employer has an active vacancy. - Distinguish verified facts from AI-generated recommendations.

The result includes the learner profile, structured advice, sources, retrieval trace, and evidence flags.

7. Python Assessor Workflow

The Python assessment implementation is in **src/agents/assessor_agent.py**.

The assessor receives: - Submitted Python code. - Assignment instructions. - Optional expected output.

The code is executed through **python_repl_tool**, which delegates execution to the Docker executor. If execution fails, the assessor returns an **Unable to verify** error result. If execution succeeds, the assessor uses Gemini structured output matching **GradingFeedback**.

The grading logic checks the submitted solution against the assignment and execution information. The implementation uses a passing threshold of 60 in the assessor.

src/tools/test_runner.py executes automated unit test assertions against the learner's code. To prevent student code from forging success reports, a cryptographically unique UUID session token wraps metric emission. If assertions fail or syntax errors occur, programmatic guardrails cap the grade and enforce a failing status.

src/main.py provides an Interactive CLI (**--mode interactive**) and an Automated Demo (**--mode demo**) driving the central LangGraph Orchestrator. The demo executes both Career Advisor and Python Assessor workflows using synthetic learner profiles (**SYN-DEMO-01**), guaranteeing zero personal data leakage, and handles Human-in-the-Loop review interruptions.

8. Docker Code Execution

The Docker configuration is in **sandbox/Dockerfile**; execution controls are implemented in **src/tools/docker_executor.py**.

The sandbox image is based on **python:3.10-slim**, creates a non-root user with UID 10001, and runs from **/sandbox**.

The runtime configuration includes:

- Network disabled.
- Read-only container root filesystem.
- Memory limit of 128 MB.
- CPU quota of 0.5 CPU.
- Process limit of 64.
- All Linux capabilities dropped.
- **no-new-privileges**.
- A temporary filesystem mounted with **noexec** and **nosuid**.
- A five-second execution timeout.
- Automatic container removal.
- Bounded output capture: stdout and stderr are each limited to 64 KiB.

Output is read in chunks by dedicated reader threads. Once a stream reaches its capture limit, the reader continues draining and discarding additional bytes so a full pipe does not block the child process. The returned result includes an **output_truncated** flag and a truncation marker when applicable. This bounds application memory used for captured output; it is not a substitute for broader host-level disk, logging, or container-runtime controls.

These controls reduce risk but do not establish a production-grade security boundary. Do not expose this prototype to arbitrary public submissions without independent security review, stronger isolation (e.g. dedicated worker hosts or microVMs), monitoring, and operational limits. Docker daemon access itself is highly privileged and must remain under trusted application control.

9. Evaluator-Optimizer

src/agents/evaluator_optimizer.py evaluates successful agent responses for quality and can request an improved response.

The workflow: 1. Evaluates the original response using a structured evaluation result. 2. If the response does not pass, generates an optimized response using evaluator feedback. 3. Repeats until the evaluator passes or the iteration limit is reached. 4. Preserves the original response if evaluation or optimization fails.

The orchestrator sets **MAX_OPTIMIZATION_ITERATIONS = 3**. The evaluator-optimizer tests use mocks, so they validate control flow without consuming Gemini API quota.

For assessment results, the orchestrator supplies the assignment, expected output, submitted code, execution result, numerical score, and pass status as protected context. The evaluator prompt is instructed not to change execution facts or grading values.

10. LangGraph Orchestration and Human Review

src/agents/orchestrator.py defines and compiles the main graph.

Routing

The router uses the explicit **request_type** first. Otherwise, submitted code plus an assignment routes to the assessor; a learner ID routes to the advisor. Unrecognized requests go to the error node.

Evaluation routing

Successful advisor and assessor results are sent to the evaluator-optimizer. Failed or unverifiable results bypass optimization and end the workflow.

Human-in-the-Loop placement review

The current prototype treats Fawry as a tier-1 employer. A human review is requested only when: - Career advice succeeded. - The verified learner profile lists Fawry as a preferred employer. - The learner's readiness status is **Ready for Placement**.

The placement review node calls LangGraph **interrupt()** with a review payload. The graph can then be resumed with a human decision using **Command(resume=...)** and the same thread/checkpoint context.

The decision records approval, reviewer, comments, learner ID, and employer review. Approval applies only to the recommendation. It does not verify a live vacancy, confirm employer acceptance, contact the employer, or submit an application.

Persistent Checkpointing

The graph is compiled with **SqliteSaver** by default, storing thread checkpoints, intermediate writes, and channel states in **data/orchestrator_checkpoints.sqlite3**. This provides persistent checkpoint storage across process restarts and supports resuming interrupted Human-in-the-Loop placement reviews.

11. Learner Database and Tools

src/tools/db_queries.py provides privacy-hardened SQLite access. The SQL query tool accepts read-only SELECT queries, opens the database in read-only mode (**mode=ro**), caps returned rows at 5 (**MAX_QUERY_ROWS = 5**), enforces an explicit column allowlist, and rejects SQL comments and internal metadata queries (**sqlite_master**, **pragma**). For retrieving individual learner profiles, **get_learner_by_id()** uses parameterized queries to enforce strict single-learner isolation and prevent cross-learner data exposure.

The database is a local runtime dependency and is not included in the public repository. This documentation intentionally omits learner IDs, names, scores, salary targets, employer preferences, and individual

readiness details. Use synthetic records for demonstrations and tests.

Do not publish real learner information unless the data owner has explicitly authorized that disclosure and the data has been reviewed for personal or confidential information.

Other tool modules include:

- **document_loader.py**: document ingestion support.
- **audio_transcriber.py**: audio transcription support.
- **rag_retriever.py**: document chunking, embedding, ChromaDB ingestion, and retrieval.
- **code_repl.py**: LangChain tool wrapper for Python execution.
- **docker_executor.py**: Docker-backed execution.

12. Knowledge Sources and Evidence Limitations

Knowledge-base inputs are maintained locally and are not published in this repository. The current ChromaDB collection contains project context, not the complete official Sprints curriculum.

Source documents may contain client or learner information. Before ingestion or redistribution, verify that the material is authorized for the intended use. AI-generated transcripts must be human-reviewed before being treated as authoritative evidence. Incomplete or unverified source material must be clearly labeled and must not be used to assert official curriculum coverage, employer hiring criteria, or current job vacancies.

The learner database does not contain a live job openings table.

13. Installation and Configuration

Prerequisites

- Python 3.10.
- Docker Desktop with the Linux container engine available.
- A Google Gemini API key for live LLM calls.

Create and activate a virtual environment

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

Install dependencies

```
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

Configure the API key

Create a **.env** file in the repository root:

```
GOOGLE_API_KEY=your_api_key_here
```

Do not commit **.env** or expose the API key in logs, screenshots,

notebooks, or source control. The project loads the key through

python-dotenv.

Build the sandbox image

Run from the repository root:

```
docker build -t sprints-python-sandbox:latest -f sandbox/Dockerfile .
```

Run the current assessor demonstration

```
python -m src.main
```

The current demonstration makes live Gemini calls during grading. If the

API quota is exhausted, it may fail even when Docker execution works.

Automated tests should use mocks and should not require a live API key.

14. Testing

The test suite contains 34 automated tests across agent workflows and tool layers:

- **tests/test_agents/test_agentic_rag.py**: validates Agentic RAG retry behavior and sufficiency stopping.
- **tests/test_agents/test_evaluator_optimizer.py**: validates iteration limits and optimization convergence.
- **tests/test_agents/test_orchestrator_evaluator.py**: validates LangGraph routing, assessment evaluator routing, and Human-in-the-Loop interrupt/resume cycles.
- **tests/test_tools/test_db_queries.py**: validates bounded query limits (max 5 rows), column allowlisting, non-SELECT rejection, multi-statement blocking, SQL comment prevention, system table protection, and parameterized single-learner isolation.
- **tests/test_tools/test_docker_executor.py**: validates bounded 64 KiB streams, empty/invalid submission handling, mock-based timeout and container cleanup, popen exception safety, real container execution, non-zero exit handling, syntax error capture, 5-second runtime timeout enforcement, massive output truncation, read-only root restriction, and network isolation.

Run the complete test suite:

```
pytest -v
```

All 34 tests run in approximately 24 seconds and pass with 100% success. Continuous Integration is automated via GitHub Actions (**.github/workflows/tests.yml**).

15. Known Limitations and Recommended Improvements

Area Current limitation Recommended next

improvement

---------- ---------- ----------

Knowledge base Project context Ingest approved

documents only; not curriculum material and

full official track document versions

curriculum

Source quality Discovery transcript is Human-review and

AI-generated and correct transcript

unverified

Kickoff deck Slides 3 and 4 are Obtain a complete
truncated source deck
Job matching No live vacancy Integrate a trusted
database jobs feed before
claiming vacancy
matches
Placement review Prototype Make employer tiers
Fawry-specific configurable and add
condition reviewer
authentication/audit
Checkpointing **SqliteSaver** local Evaluate distributed
SQLite storage; single checkpointers (Postgres)
node deployment for distributed clusters
Docker execution Prototype sandbox with Evaluate microVM/gVisor
64 KiB output limit isolation and external
and resource bounds; security audit before
not certified for accepting arbitrary
untrusted public code untrusted public code
Test coverage Comprehensive agent & Add live end-to-end API
tool test suite integration benchmarks
(47 tests passed; CI)
User interface CLI only (`--mode Build a web dashboard
interactive/demo`) (FastAPI/React) for
students & advisors
Observability Console/logging support Add structured traces,
is basic request IDs, and
privacy-aware logs

---

16. Conclusion

The current implementation demonstrates a multi-agent architecture with a LangGraph router, a retrieval-grounded career advisor, a Docker-backed Python assessor, an evaluator-optimizer loop, and a conditional human approval checkpoint.

The project has a working prototype structure and a reported passing test suite. Its key constraints are the limited project-context knowledge base, lack of live vacancy data, in-memory checkpointing, and the need for further sandbox hardening and tool-level tests. These limitations should remain explicit in any demo, report, or presentation.